

Data Science Introduction

Neural Networks

Nistor Grozavu

nistor.grozavu@cyu.fr

Note. Le TP doit être effectué en Python en utilisant numpy et scikit-learn. Préparez un rapport.

A) Régression avec un Perceptron Multi-Couches

1. Créez un ensemble de points (une 20^{ème}) suivant une fonction sinusoïde avec un bruit gaussien d'écart type 0,2. On va essayer de retrouver la fonction à partir des points avec un Perceptron Multi-Couches (MLP). Analyser la documentation sur MLP dans la librairie scikit-learn et analysez toutes les paramètres.

Vous pouvez utiliser :

```
x = np.linspace(0,8,20).reshape(-1,1)
```

```
y = np.sin(x) + np.random.randn(20)*0.2
```

2. Initialisez le MLP avec la fonction *MLPRegressor* de scikit-learn. On veut 3 neurones cachés, une fonction d'activation linéaire et un pas d'apprentissage (learning rate) de 0,1 avec un nombre d'itérations de 100. Ces sont des paramètres à fixer dans la fonction MLP.
3. Lancez l'apprentissage du MLP. Utilisez la fonction *fit*.
4. La fonction *predict* permet d'utiliser le MLP appris pour faire de la régression.
5. Tracez sur une même figure les points d'apprentissage, la fonction sinusoïde réelle et celle estimée par le MLP en utilisant la fonction *plot()*.

B) Classification avec un Perceptron Multi-Couches

1. Les données IRIS sont constituées de 4 colonnes qui correspondent à 4 variables de description. La 5^{ème} colonne donne la classe de chaque objet. Transformez ces données en deux matrices contenant les données et le label (target).
2. Pour permettre l'apprentissage par un MLP, il faut « binariser » les numéros de classe.
3. Initialisez le MLP avec la fonction *MLPClassifier*. On veut 2 neurones cachés, une fonction d'activation logistique et un pas d'apprentissage de 0,2.
4. On garde les mêmes options que dans l'exercice précédent. Lancez l'apprentissage du MLP avec la fonction *fit*. On veut utiliser l'algorithme d'optimisation *de descente de gradient stochastique (lbfgs)*.

5. Affichez les résultats désirés et les résultats obtenus dans une même figure. Vous pouvez utiliser un ACP par exemple. Chaque point aura une couleur différente selon sa classe. Les résultats sont-ils bon ?
6. Essayez de normaliser les données (centrées – réduites) en utilisant la fonction `scale()`. Cela améliore-t-il la classification ?
7. Essayez de faire varier les paramètres du MLP (nombre de neurones cachés, etc...). Comment ces paramètres influencent les résultats ?